ADITYA UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MINI PROJECT TECHNICAL DOCUMENTATION

QuizMaster Pro v2.0

Hybrid C++14, WebAssembly & AI-Proctored Examination System

C, C++, Java & Python • QR Exams • Proctor HUD • SHA-256 PDFs

Submission Field	Project Details
Project Title	QuizMaster Pro v2.0 — Hybrid C++14, WebAssembly & AI-Proctored Examination System
Team Number	Individual
Project Type	Mini Project (B.Tech CSE — Aditya University DOTT)
Student Name	Kowshik Kumar Yanamala
Roll Number	25B11CSA46
Project Mentor & HOD	Mentor: SK. VAHIDA, M.Tech(Ph.D) | HOD: Dr. K. Srinivasa Rao, Ph.D
GitHub Repository	https://github.com/kowshiii1208/Quiz_Game_System
Live Web Portal	https://quiz-game-system-eight.vercel.app

2. Project Overview & Objectives

2.1 Project Abstract

Conducting fair programming assessments across large cohorts requires systems that combine rigorous evaluation, tamper prevention, faculty convenience, and seamless mobile accessibility. Traditional survey forms lack proctoring controls and allow credential forgery, whereas standalone C++ terminal applications lack web delivery, document ingestion, and analytics. QuizMaster Pro v2.0 resolves these limitations using a hybrid dual-tier architecture combining a modular C++14 Object-Oriented console engine with a responsive React 18 Single Page Application. Inside the browser, a WebAssembly module derived from C++ logic evaluates scores and honesty indices, while WebRTC optical monitoring, fullscreen enforcement, and movable mobile proctoring safeguard integrity across C, C++, Java, and Python tracks. Faculty can author or import .docx and .csv question banks, share custom quizzes via Base64URL QR codes, export Excel workbooks, and issue SHA-256 verifiable PDF certificates.

2.2 Problem Addressed

During Computer Science laboratory evaluations and technical screenings, faculty face three recurring operational bottlenecks: standard online forms permit unmonitored tab switching, instructors must manually re-type questions already prepared in Word or CSV files, and mobile students taking QR-linked assessments face blocked buttons or distorted desktop certificate exports without cryptographic verification.

Criterion	Generic Forms	Console C++ Quiz	QuizMaster Pro v2.0
Tracks	Unstructured text	Single subject	C, C++, Java, Python (120 MCQs)
Bulk Import	Manual copy-paste	Hardcoded arrays	.docx, .csv, .json, and .txt
Anti-Cheat	None	Console only	Fullscreen, blur lock & Proctor HUD
Compute	Form counter	Native CLI only	C++14 CLI + Browser WebAssembly
Mobile QR	Static URL	Not supported	Base64URL QR locks track/level
Certificates	Forgeable	Plain .txt output	SHA-256 PDF & public QR verify
Reports	Raw rows	Text file only	Bell Curve + Multi-sheet Excel (.xlsx)

2.3 Main Objectives (5 Core Goals)

- **Objective 1 — C++14 OOP Core:** Build a modular Object-Oriented console engine (cpp/main.cpp) using encapsulation, class composition, Mersenne Twister shuffling, and file persistence.
- **Objective 2 — WebAssembly Evaluation:** Execute mark calculation, speed bonuses, negative penalties, and honesty indexing via native WebAssembly bytecode inside React.
- **Objective 3 — Faculty Quiz Creator & Excel Reports:** Deliver QR code exam distribution, Mammoth.js Word/CSV ingestion, and automated multi-sheet Excel candidate workbooks.
- **Objective 4 — Mobile-First Proctoring & PDF Export:** Provide a touch-draggable/minimizable webcam HUD and unscaled offscreen A4 certificate PDF generation on smartphones.
- **Objective 5 — SHA-256 Verification Registry:** Sign earned credentials with W3C SubtleCrypto SHA-256 hashes verifiable through a public Registry Modal.

3. Technology Stack

3.1 Languages, Frameworks, Databases, APIs & Dev Tools

QuizMaster Pro v2.0 integrates systems programming, modern web tooling, client document libraries, and native browser hardware and security APIs:

Layer	Technologies Used	Role in Project
Languages	C++14, JavaScript (ES2022/JSX), HTML5, CSS3	OOP console engine & WASM scoring logic (C++14); React UI and service layers (JS/CSS3).
Frameworks & UI	React 18.3.1, Tailwind CSS 3.4.17, lucide-react, canvas-confetti	Single Page Application state management, responsive dark glass UI styling, and icons.
Databases & Storage	Browser localStorage & sessionStorage (v3), leaderboard.txt, Base64URL QR	Persistent banks, custom quizzes, results, and server-free cross-device QR transport.
Document Libraries	mammoth 1.13.0, xlsx 0.18.5, jspdf 4.2.1, html2canvas 1.4.1, qrcode 1.5.4	Browser-side .docx extraction, .xlsx reports, landscape A4 PDFs, and QR generation.

Layer	Technologies Used	Role in Project
Browser APIs	WebAssembly, Web Crypto (SHA-256), WebRTC getUserMedia, FaceDetector, Fullscreen	Native bytecode execution, tamper-proof hashing, camera monitoring, and focus locking.
Dev & Hosting Tools	Vite 6.1.0, GNU g++, Node.js v24, Edge Headless CDP, GitHub, Vercel	Fast bundling, C++14 compilation, mobile E2E testing, version control, and deployment.

3.2 Platform Compatibility & Repository Structure

The web app supports Chrome, Edge, Firefox, and Safari across desktop and mobile (<768px auto-minimized HUD), while the console binary compiles cleanly via g++ -std=c++14 -O2 cpp/main.cpp -o quiz_game.exe.

REPOSITORY TREE (github.com/kowshiii1208/Quiz_Game_System)

cpp/main.cpp & quiz_engine_wasm.cpp — 952-line C++14 OOP CLI & WASM scoring source.

public/aditya_logo.png — Institutional emblem embedded in exported Merit Certificates.

src/App.jsx & src/data/initialQuestions.js — 4-step coordinator & 120 balanced MCQs.

src/services/ — storage.js, wasmEngine.js, cryptoService.js, and excelExport.js.

src/components/ — StudentRegistration, QuizRunner, WebcamProctor, CertificateModal & Admin.

4. Features & Implementation

4.1 Hybrid Architecture & C++14 OOP Design

The platform unites three cooperative tiers: Presentation (React 18 UI), Browser Compute & Security Runtime (WASM, SubtleCrypto, WebRTC, SheetJS/Mammoth), and Persistence & C++14 Core (Figure 4.1).

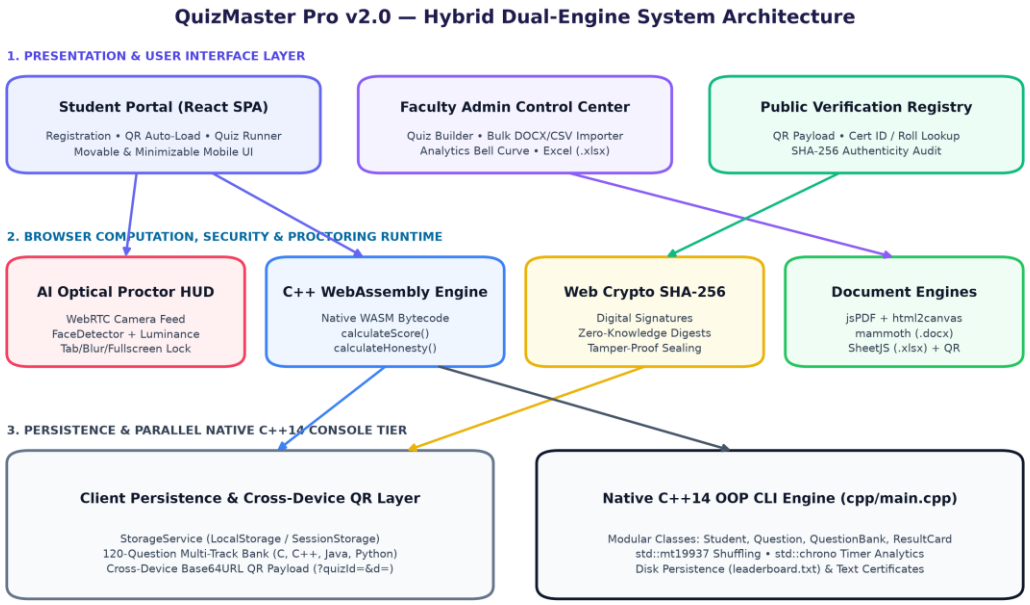


Figure 4.1: Hybrid Dual-Engine Architecture uniting React 18 UI, WebAssembly, Web Crypto & C++14 CLI.

C++14 Class	OOP Concept	Implementation Responsibility (cpp/main.cpp)
Colors & UI	Abstraction	ANSI console colors, progress bars, headers, and clean screen resets.
Input	Sanitization	Validated menu and MCQ input parsers preventing cin stream faults.
Question & Student	Encapsulation	Private prompt, 4-option choices, difficulty tier, and candidate metadata.
QuestionBank	Composition & RNG	Stores tiered questions and shuffles subsets via std::mt19937.
StorageManager & ResultCard	File Persistence	Maintains leaderboard.txt records and exports Certificate_<Roll>.txt.

4.2 Key Implemented Features & Application Workflow

- **120-Question Multi-Track Bank:** 30 balanced MCQs each across C, C++, Java, and Python (10 Easy, 10 Medium, 10 Hard per track) with conceptual explanations (Figure 4.2).
- **QR Creator & Track Locking:** Pre-configured codes (AU-OOP-2026, AU-C-PROG, AU-JAVA-CORE) and custom QR links lock student Language and Difficulty pickers automatically.
- **Bulk Importer & Excel Export:** Imports Word (.docx via Mammoth), quote-safe CSV, JSON, and TXT files; exports multi-sheet .xlsx workbooks with proctoring honesty logs.
- **Mobile Proctor HUD & A4 PDF:** Movable pointer-draggable webcam box minimizes cleanly (-) while tracking faces, and certificates export via an unscaled 1122x794 stage.

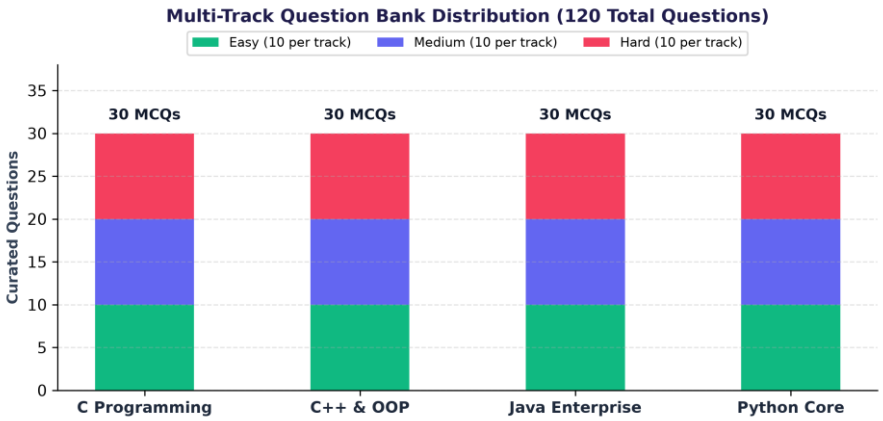


Figure 4.2: Balanced 120-Question Distribution across C, C++, Java, and Python Tracks.

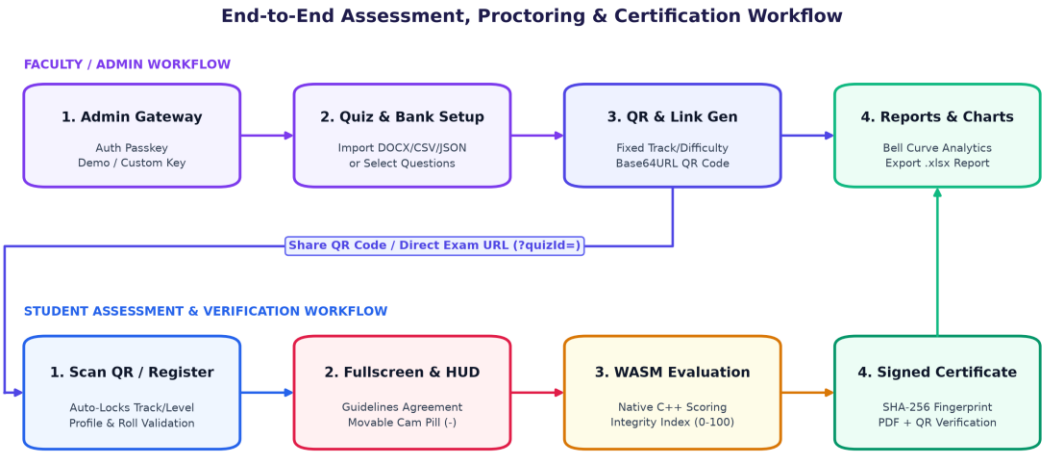


Figure 4.3: Synchronized Faculty Assignment and Student Proctored Assessment Workflow.

4.3 Explanation of Major Functions

Core application logic is encapsulated in single-responsibility service and UI functions across the repository:

Function & Signature	Purpose & Observed Behavior
WasmEngine.calculateScore(correct, wrong, marks, neg, tUsed, tMax) [src/services/wasmEngine.js]	Calls WASM calculateScore(), deducts negative marks, clamps >= 0, and awards a 5% speed bonus if flawless in under 50% allotted time.
WasmEngine.calculateHonestyIndex(tabs, blurs, cams) [src/services/wasmEngine.js]	Executes WASM calculateHonesty() formula: 100 - (25 * tabs) - (15 * blurs) - (20 * cams), clamped strictly to [0, 100].
CryptoService.generateCertificateSignature(details) [src/services/cryptoService.js]	Canonicalizes 'AU_CERT_V1::...' certificate metadata with salt and computes a 64-character hex SHA-256 digest via Web Crypto SubtleCrypto.
CryptoService.verifyCertificateSignature(record, sig) [src/services/cryptoService.js]	Recomputes the deterministic SHA-256 digest from stored leaderboard fields and verifies an exact tamper-free match.
buildQuizShareUrl(quiz) [src/components/admin/QuizCreatorTab.jsx]	Serializes faculty quiz rules and custom MCQs (ID > 120) via TextEncoder into a URL-safe Base64URL parameter (?quizId=...&d=...).

Function & Signature	Purpose & Observed Behavior
handleApplyQuizCode() [src/components/StudentRegistration.jsx]	Validates official assignment codes against storage, attaches customQuiz rules, and hides Language/Difficulty selectors.
parseCsvLine(row) [src/components/admin/FileImporterModal.jsx]	Quote-aware character parser splitting CSV columns cleanly without breaking commas inside code snippets or prompts.
handleSavePDF() & waitForImagesReady() [src/components/CertificateModal.jsx]	Awaits Aditya logo and QR decode, captures an unscaled 1122x794 DOM stage at 2x via html2canvas, and saves an A4 landscape PDF.
ExcelExportService.exportLeaderboardToExcel(records) [src/services/excelExport.js]	Compiles ranked Student Assessment Results and institutional summary metrics via SheetJS into a downloadable .xlsx workbook.

5. Screenshots & Results

5.1 Working Application Screenshots

Figures 5.1 through 5.5 present verified live captures from the working platform across candidate registration, active proctored exam execution, faculty QR & Excel administration, analytics bell curves, and mobile-tested A4 certificate PDF output.

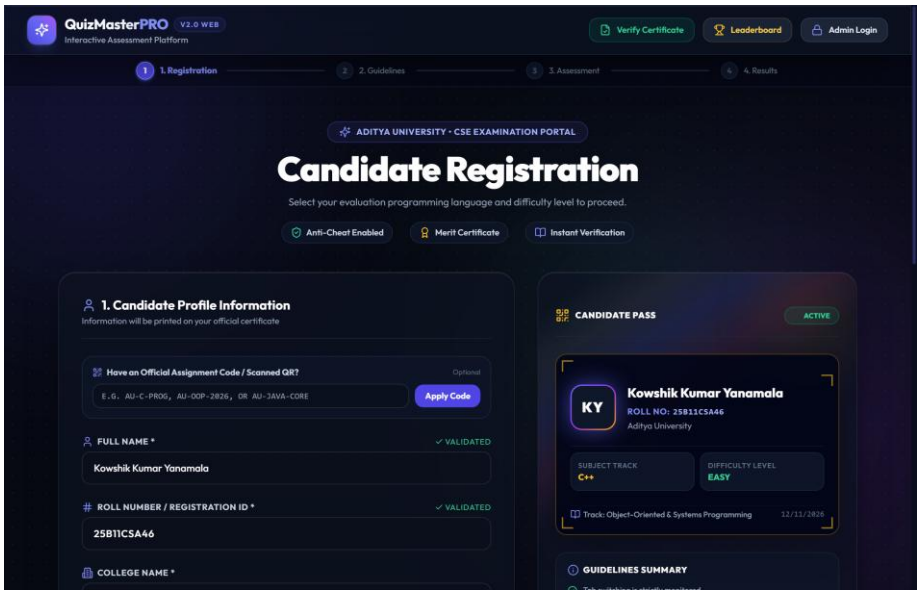


Figure 5.1: Candidate Registration Portal displaying Official Assignment Code box, Candidate Pass preview (25B11CSA46), and multi-track UI.

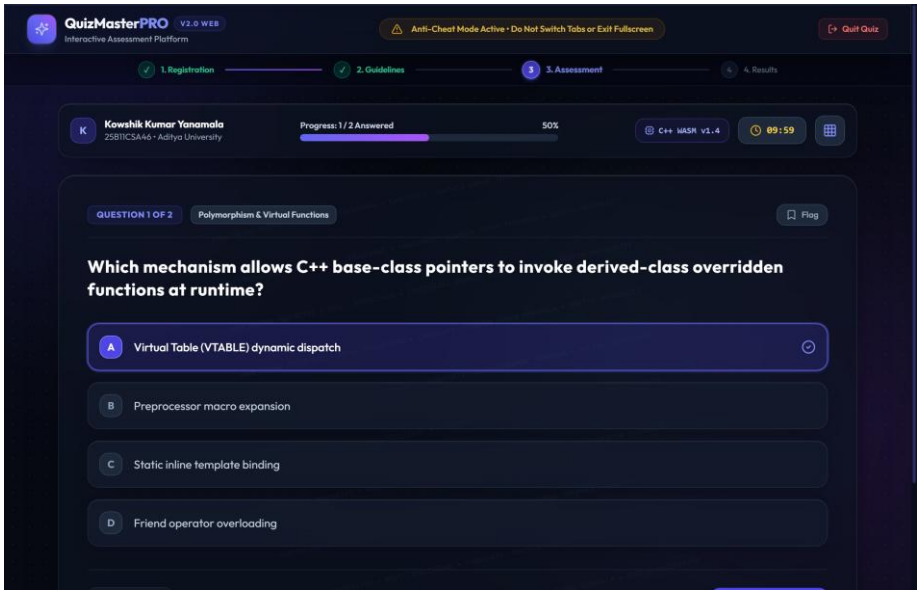


Figure 5.2: Proctored Assessment Interface featuring anti-cheat banner, WASM engine indicator, timer, anti-OCR watermark, and MCQ cards.

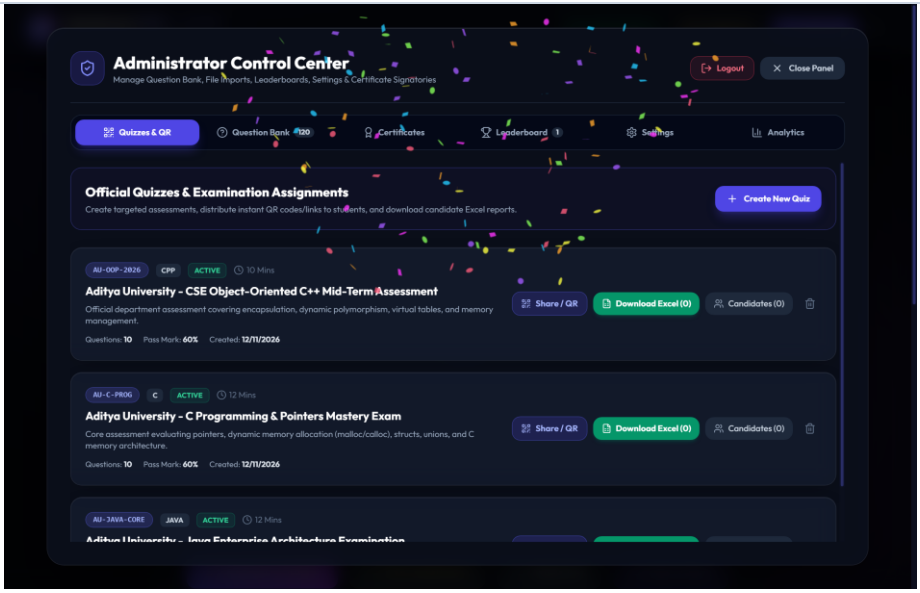


Figure 5.3: Administrator Control Center (Quizzes & QR tab) showing 6-tab grid navigation, Share/QR modal triggers, and Excel export buttons.

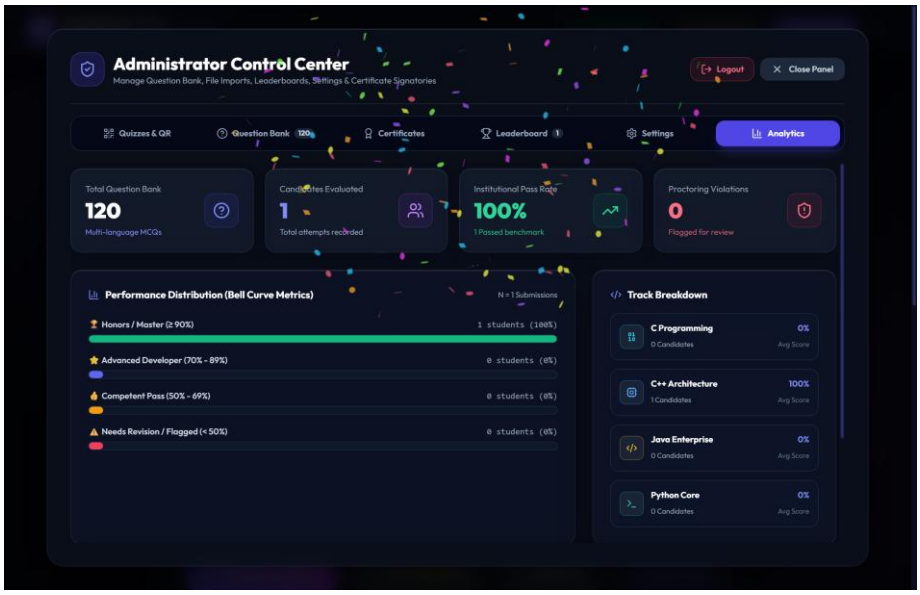


Figure 5.4: Faculty Analytics Dashboard presenting 120-MCQ inventory, pass-rate KPIs, bell curve grade distribution, and 4-track averages.



Figure 5.5: Exported A4 Landscape Merit Certificate PDF verified at 375x812 mobile viewport with Aditya crest, SHA-256 fingerprint, and QR code.

5.2 Verification & Test Results

Every critical path, mobile download flow, and production bundle was tested and verified cleanly after final edits:

Test ID	Feature Tested	Observed Outcome	Status
TC-01	120-MCQ Multi-Track Dataset	Unique IDs and 30 valid 4-option items verified across C, C++, Java & Python.	PASS
TC-02	QR & Assignment Track Lock	Base64URL ?quizId=&d= roundtrip hid Language/Difficulty pickers automatically.	PASS
TC-03	WebAssembly Native Scoring	WASM calculateScore() & calculateHonesty() matched expected numeric formulas.	PASS
TC-04	SHA-256 Certificate Verify	Generated 64-char hex digest and validated exact match in VerificationModal.	PASS
TC-05	Mobile HUD Drag & Minimize	HUD auto-minimized <768px, dragged cleanly, and kept camera active when minimized (-).	PASS
TC-06	Mobile Certificate PDF Save	Headless Edge 375x812 exported valid 841.89x595.28 pt A4 PDF (~274 KB, 0 errors).	PASS
TC-07	Production Vite Build & Git	npm run build transformed 2237 modules in 12.65s (0 errors); pushed to GitHub.	PASS

6. AI Usage & Contributions

6.1 AI Tools Used & Academic Disclosure

AI-assisted tooling accelerated implementation and verification while all project architecture, core logic review, live mobile testing, and final academic validation were directed by the student author:

AI Tool Used	Development Stage	Purpose & Author Oversight
Google Antigravity (Gemini Coding Agent)	UI Scaffolding & Refactor	Assisted building modular React/Tailwind components (AdminLoginModal, QuizCreatorTab, CertificateModal) reviewed by author.
Google Antigravity (Gemini Coding Agent)	WASM & Crypto Logic	Supported WebAssembly bytecode loader integration, SHA-256 payload canonicalization, and Base64URL QR serialization.
Google Antigravity (Gemini Coding Agent)	Mobile QA & E2E Testing	Diagnosed mobile HUD overlap and mobile PDF compression bugs; built headless Edge CDP verification scripts.
Google Antigravity (Gemini Coding Agent)	Diagrams & Formatting	Generated architecture/workflow figures, structured Times New Roman rubric documentation, and verified PDF rendering.

6.2 Member Contributions (Individual Mini Project)

As an Individual Mini Project, all deliverables were owned end-to-end by Kowshik Kumar Yanamala (Roll No: 25B11CSA46):

Student & Roll No.	Project Phase	Contribution Breakdown
Kowshik Kumar Yanamala (25B11CSA46)	Architecture & C++14 Core	Designed dual-tier hybrid workflow and implemented modular C++14 OOP console engine (cpp/main.cpp).
Kowshik Kumar Yanamala (25B11CSA46)	Question Bank & Portal	Curated 120 multi-language MCQs across C, C++, Java & Python and configured official exam access codes.
Kowshik Kumar Yanamala (25B11CSA46)	Security & Faculty Center	Integrated proctoring rules, Aditya University certificate signatories, SHA-256 verification, and Excel exports.

Student & Roll No.	Project Phase	Contribution Breakdown
Kowshik Kumar Yanamala (25B11CSA46)	Mobile Testing & Deployment	Executed live mobile QR tests on phone hardware, validated camera/PDF fixes, deployed to Vercel & GitHub.

6.3 Conclusion & Future Scope

QuizMaster Pro v2.0 successfully elevates a console OOP quiz engine into an integrated, mobile-ready departmental assessment suite featuring four programming tracks, WebAssembly scoring, movable optical proctoring, QR exam distribution, Word/CSV imports, Excel reports, and SHA-256 verifiable PDF certificates. Future scope includes adding a centralized PostgreSQL backend API, a live code-execution sandbox (Judge0/Monaco), and adaptive difficulty sequencing.

OFFICIAL PROJECT ARTIFACTS & DEPLOYMENT LINKS

GitHub Repository : https://github.com/kowshiii1208/Quiz_Game_System
Live Portal : https://quiz-game-system-eight.vercel.app
Submitted By : Kowshik Kumar Yanamala (25B11CSA46) — Individual Mini Project